Introducción

Definición	La normalización es una secuencia de pasos mediante los cuales se crea y mejora un modelo de base de datos relacional.

Los pasos involucrados en el proceso de normalización se los denomina Formas Normales. Estas formas normales, aplicadas secuencialmente son las que permiten la creación y la mejora de un modelo de base de datos.

Como dijimos, la normalización es un proceso que involucra una serie de 6 pasos que van desde la primera forma normal a la sexta forma normal. Por lo general sólo se implementan los primeros 3 pasos, es muy raro encontrar modelos que implementen alguno de los otros niveles.

El objetivo principal de la normalización es eliminar la duplicación de datos y así minimizar la información redundante. El resultado de este proceso, se puede decir que, es una mejor organización de los datos y un mejor aprovechamiento del espacio físico.

Anomalías

En las bases de datos relacionales se trata de eliminar las anomalías. Las anomalías son un efecto no deseado desde el punto de vista de la gestión de los datos ya que pueden conducir a que los mismos se corrompan, provocando a futuro pérdida de información o lo que es peor, información errónea. Existen varios tipos de anomalías:

Insert

Producida cuando se inserta un registro a una tabla hija sin que exista el registro relacionado en la tabla padre.

Delete

Producida cuando se elimina un registro de una tabla padre dejando huérfanos uno o varios registros en tablas hijas.

Update

Al igual que en el caso de *Delete*, tanto la tabla padre como las tablas hijas deben ser consideradas durante la operación para evitar la aparición de registros huérfanos.

Formas Normales

Las formas normales son aplicadas a las tablas de una base de datos. Decir que una base de datos está en la forma normal N es decir que todas sus tablas están en la forma normal N.

Como mencionamos anteriormente las bases de datos relacionales se normalizan para:

- Evitar la redundancia de los datos.
- Eliminar inconsistencias generadas al actualizar y/o eliminar datos de las tablas.
- Proteger la integridad de los datos.

Primera Forma Normal (1FN)

Principalmente consiste en eliminar los grupos de campos repetidos, creando nuevas tablas para dichos campos, estableciendo una relación 1 - N entre la tabla original y las nuevas.

Los pasos para lograr la 1FN son:

- Eliminar los grupos de campos repetidos
- Definir claves primarias
- Todos los registros de todas las tablas deben ser identificables por la clave primaria.
- Todos los campos, excepto los de la clave primaria, deben depender de la clave primaria.
- Crear nuevas tablas para mover los grupos de campos repetidos de la tabla original.

Ejemplo:

a.

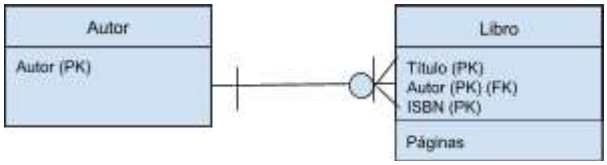
Autor	Título_01	Páginas_01	Título_02	Páginas_02	ISBN_01
Tolkien	El señor de los anillos	1000	El Hobbit	300	1

b.

Autor	Títulos	Páginas	ISBNs
Tolkien	El señor de los anillos, El Hobbit	1000, 300	1, 2

Tanto en el ejemplo ‘a’ como ‘b’, vemos que existen grupos de campos para el *título* y la *cantidad de páginas*.

Para llevar esta tabla, que se encuentra en 0FN, a 1FN debemos eliminar los grupos de campos y buscar y generar las claves primarias correspondientes. Es probable (casi una certeza) que surjan nuevas tablas en este paso



Si en cualquiera de los ejemplos mencionados previamente existieran, por ejemplo, datos de la "Editorial" del libro, esos datos deberían ser extraídos en una nueva tabla ya que los mismos no son datos que dependan de la clave primaria del libro.

Segunda Forma Normal (2FN)

Como precondición, la tabla debe estar en 1FN.

Mientras que en el paso anterior, se eliminaban los campos repetidos, en este paso, el objetivo principal es eliminar los valores estáticos (repetidos) y pasarlos a nuevas tablas estableciendo relaciones N - 1 entre la tabla nueva y la tabla original.

Desde el punto de vista matemático de la relación, una relación 1-N o N-1 son exactamente iguales lo que cambia en este paso es el enfoque y el objetivo final que es eliminar los datos duplicados.

Otro de los objetivos logrados es que las nuevas tablas, obtienen una clave primaria conformada por un único campo.

Los pasos para lograr la 2FN son:

- La tabla debe estar en 1FN
- Todos los campos que no son parte de la clave primaria deben poder ser determinados a partir de la clave primaria (dependencia funcional)
- Eliminar las dependencias parciales. Una dependencia parcial se da cuando un campo depende parcialmente de la clave principal.
- Crear nuevas tablas para mover los campos que dependen parcialmente de la clave principal.

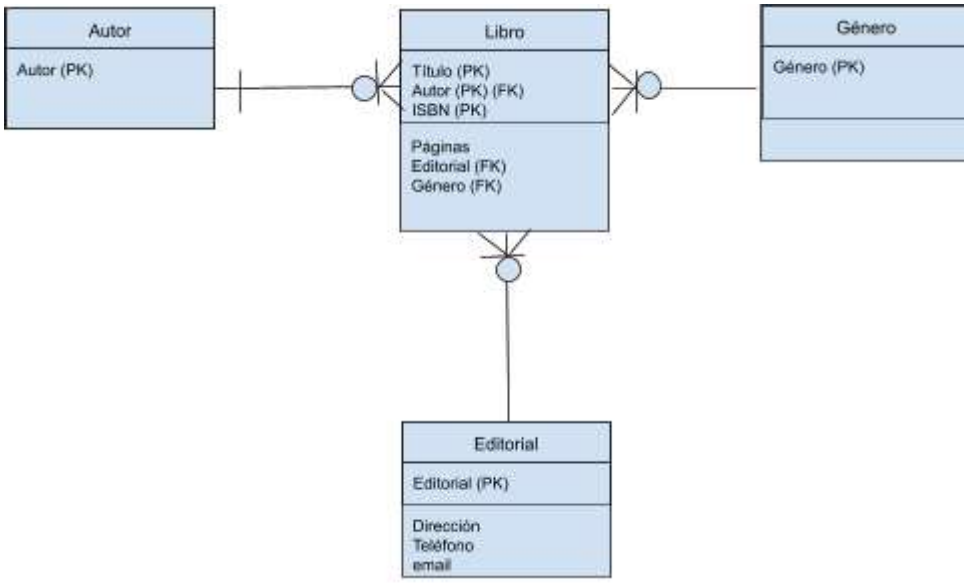
Ejemplo:

Supongamos que en las tablas del modelo de libros tenemos además datos sobre la editorial y sobre el género del libro

Autor	Título	ISBN	Páginas	Editorial	Dirección	Género
Tolkien	El señor de los anillos	1	1000	Cosme Fulanito	Calle Siempre Viva 1000	Ciencia Ficción

En esta tabla vemos que tanto los datos del Editorial (campos Editorial y Dirección) como el Género, no son datos que tengan una dependencia directa con la clave primaria, estos son los campos candidatos a remover y trasladar a nuevas tablas.

Tercera Forma Normal (3FN)



Como precondición, la tabla debe estar en 2FN.

En este paso se trata de eliminar las dependencias transitivas, que se dan cuando un campo tiene una dependencia funcional con otro campo que no forma parte de la clave primaria.

Básicamente todos los campos que no formen parte de la clave primaria deben depender directamente de la clave primaria.

Casos

Relaciones N - M. Para lograr que las tablas pasen a estar en 3FN, hay que generar una tabla intermedia que permita obtener registros y atributos particulares sin repetición.

Otro caso es cuando se tienen datos en común entre 2 o más tablas. Por lo general vamos a encontrar una dependencia transitiva y esta debe ser eliminada. Los campos comunes deben ser movidos a una nueva tabla.

Como se mencionó en la introducción, las dependencias transitivas deben ser eliminadas moviendo los campos involucrados hacia una nueva tabla.

Por último si existen campos calculados, los mismos deberían[1] ser eliminados de las tablas.

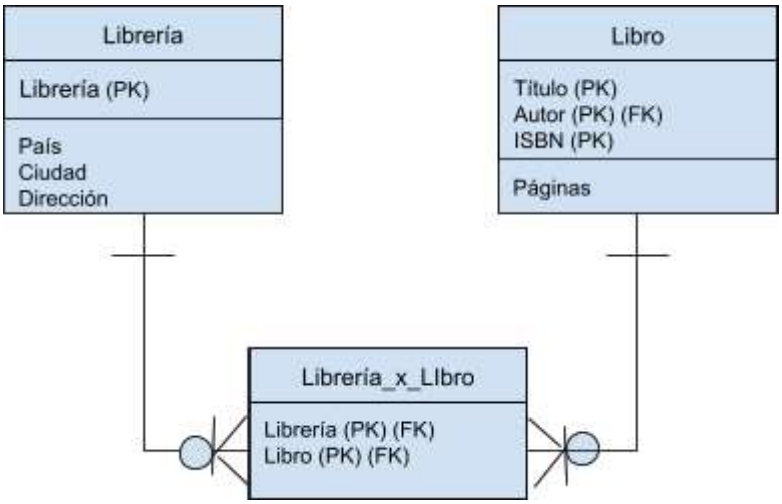
Ejemplo

Supongamos que a nuestro modelo se agregan las librerías, y necesitamos saber qué libros están en cada librería, podríamos tener en primera instancia:

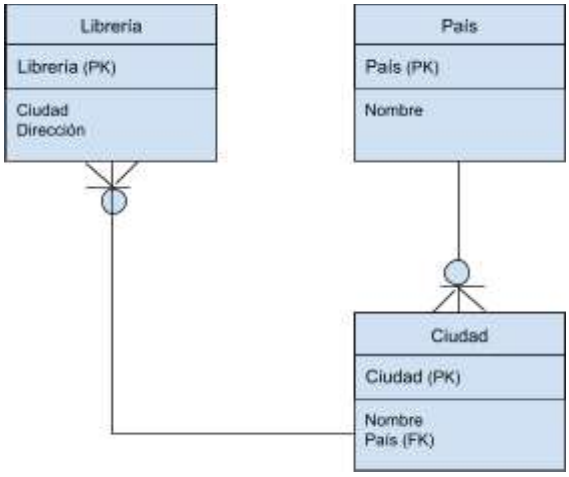
Librería	País	Ciudad	Dirección	Libro
La nueva	Argentina	CABA	xxx 123	Object Oriented Análysis and Design
La nueva 2	Argentina	CABA	xxx 123	Clean Code
El rincón del libro	Argentina	CABA	yyy 123	Object Oriented Análysis and Design
El rincón del libro	Argentina	CABA	yyy 123	Clean Code

Claramente tenemos una relación N-M y esta tabla no se encuentra en 3FN por motivos que ya mencionamos anteriormente. La solución será:

[1] Muchas veces, por cuestiones de rendimiento no está mal tener campos calculados en las tablas.



Como vemos este modelo cuenta con otros puntos a atacar como son las dependencias parciales y las dependencias transitivas



En este último modelo eliminamos las dependencias parciales de la librería con el país y la ciudad y además la dependencia transitiva entre la ciudad y el país.

Cuarta, Quinta y Sexta Forma Normal

Además de las descritas en este apunte, existen 4 formas normales más que son

Boyce-Codd (BCNF)

Como precondición, la tabla debe estar en 3FN.

El objetivo de este paso es el de separar todas las claves candidatas en nuevas tablas. Una clave candidata es un campo o grupo de campos que podrían ser la clave primaria de la tabla.

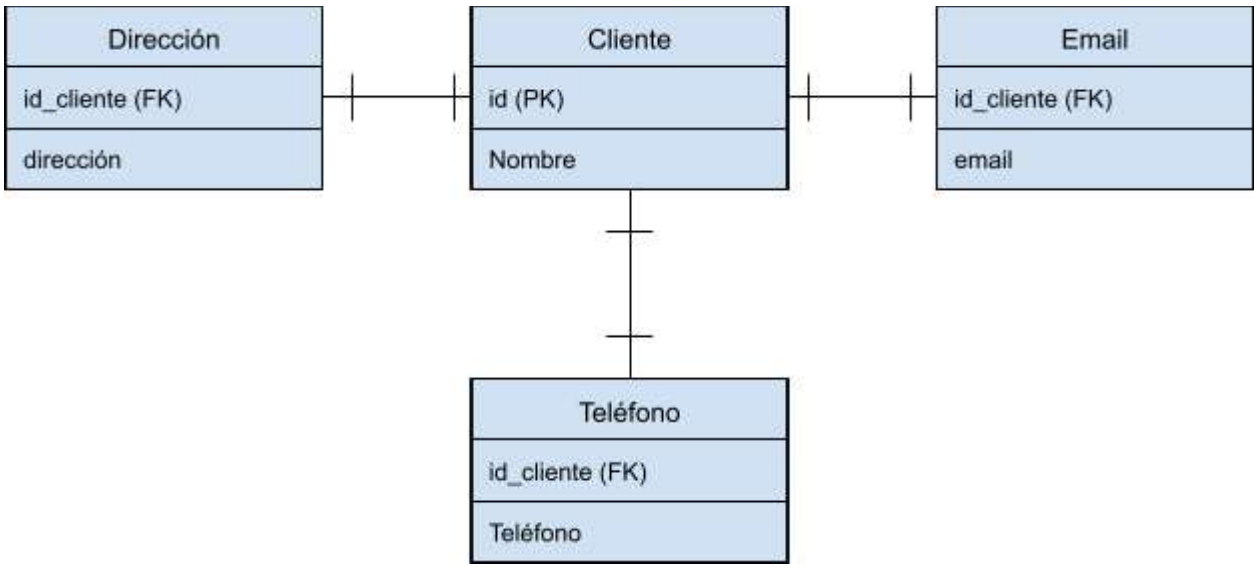
Este es un caso especial de la 3FN, ya que si sólo existe 1 clave candidata la 3FN y BCNF son iguales.

En caso de que existan varias claves candidatas, deben ser llevadas a nuevas tablas. El problema de este enfoque es que se degrada el rendimiento de las consultas.

Ejemplo

Cliente
id (PK)
Nombre Dirección Teléfono Email

Como la dirección, si contemplamos el código postal; el teléfono, si contemplamos el código de país y área; y el email son campos que poseen valores únicos, cualquiera de esos tres datos serían posibles claves primarias en sí mismas, por lo tanto la relación *Cliente* para estar en BCNF debe convertirse en 3 relaciones 1 - 1 de la siguiente manera:



Como ventajas de este enfoque podemos decir que permite eliminar campos con valores nulos (valor null). Por ejemplo, si un cliente no posee *teléfono* en la tabla original, ese campo quedará

vacío, es decir, con valor null, mientras que en el modelo BCNF no existiría el registro en la tabla *Teléfono*.

Pero también hay que tener en cuenta el rendimiento de las consultas ya que en el modelo original, para obtener todos los datos de 1 cliente sólo hay que acceder a una tabla, mientras que en el segundo hay que acceder a 4 tablas.

Cuarta Forma Normal (4FN)

Como precondición, la tabla debe estar en 3FN o BCNF.

Transforma los campos que poseen dependencias de valores múltiples en dependencias funcionales de forma tal que todos los campos tengan una dependencia funcional con la clave primaria de la tabla.

Este caso se da cuando en una tabla tenemos campos que poseen múltiples valores (separados por coma, por ejemplo) que no están relacionados uno con el otro. Estos deben ser llevados a nuevas tablas y relacionados con la tabla original.

Por lo general esto dará origen a múltiples relaciones 1 - N o N - M

Ejemplo

Empleados
Legajo (PK)
Nombre Habilidades Certificaciones

Legajo	Nombre	Habilidades	Certificaciones
1	Juan Perez	Analista, Programador	Zend PHP, C++
2	Ana Martinez	Project Manager, Tester, Analista	PMI, Testing X

En este caso hay que transformar la tabla *Empleados*, generando 2 relaciones N-M una para las Habilidades y otra para las Certificaciones. Obviamente habrá que crear las tablas intermedias que relacionen un empleado con sus habilidades y otra con sus certificaciones.

Básicamente, 4FN separa conjuntos de datos en registros, haciendo que cada registro individual sea más fácil de acceder cuando se realiza una búsqueda de coincidencia exacta.

Quinta Forma Normal (5FN)

Como precondición, la tabla debe estar en 4FN.

La 5FN también es conocida como *Quinta Forma Normal de Proyección-Unión*.

En este paso se trata de eliminar las dependencias cíclicas que pueda tener una tabla y de reducir la redundancia de los datos.

Una dependencia cíclica en el contexto de bases de datos sucede cuando la clave primaria de una tabla está compuesta por 3 o más campos y esos campos están relacionados de a pares entre sí.

El término *Proyección* se debe a que la tabla original se separa en varias tablas que contienen subconjuntos de datos (proyección) y el término *Unión* a que al realizar una consulta de todas las tablas se debe obtener el mismo resultado que se exhibía en la tabla original (join).

Ejemplo

Empleados
Proyecto (PK) Empleado (PK) Director (PK)
...

Proyecto	Empleado	Director
Análisis	X	A
Análisis	Y	B
Análisis	Z	B
Diseño	W	C
Diseño	X	C

Algunos de los problemas que podemos ver en este caso son la redundancia de datos, el hecho de que al insertar un nuevo dato habrá que hacerlo tantas veces como combinaciones se necesiten.

La 5FN nos propone generar tablas de proyección para eliminar esta redundancia de datos. La forma de hacerlo es generar 1 tabla por cada par de campos relacionados, en este caso tendremos 3 tablas:

- Proyecto-Empleado
- Proyecto-Director
- Empleado-Director



Sobre el DER del ejemplo, las líneas que relacionan las tablas no son FK, sino que simplemente están expresando una semi-relación entre las mismas.

En la práctica ésta FN no es muy utilizada debido a que se generan muchas tablas y se puede degradar el rendimiento de las consultas

Sexta Forma Normal (DKNF)

Domain Key Normal Form (DKNF) es a menudo un estado conceptual de un modelo de base de datos relacional en oposición a un proceso de transformación final. DKNF es la última forma normal y esencialmente describe cómo debería aparecer un modelo de base de datos completamente normalizado:

- No puede haber anomalías de inserción, cambio o eliminación de datos. En otras palabras, cada registro en la base de datos debe ser accesible directamente de todas las maneras, de modo que no se produzcan errores.
- Cada registro en cada tabla debe ser identificable de manera única y estar directamente relacionado con la clave principal en su tabla. Esto significa que cada campo en cada tabla está directamente determinado por la clave primaria en su tabla.
- Toda validación de datos se realiza dentro del modelo de base de datos. En lo que respecta al rendimiento de la aplicación y de la base de datos, esto es casi siempre un estado extremadamente indeseable en un entorno comercial. Es mejor dividir la

funcionalidad entre la base de datos y las aplicaciones. Algunos pueden llamar a esto reglas comerciales. Por lo general, es de conocimiento común que algunas implementaciones de reglas comerciales a menudo se dividen de manera más efectiva entre la base de datos y las aplicaciones.

Bibliografía

- Gavin Powell. Beginning Database Design. Indianapolis: Wiley Publishing, Inc, 2006. ISBN-13: 978-0-7645-7490-0.
- Abraham Silberschatz; Henry F. Korth; S. Sudarshan. FUNDAMENTOS DE BASES DE DATOS. Cuarta Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A. U, 2002.
- C. J. Date. Introducción A Los Sistemas De Bases De Datos. Séptima Edición. Méjico: Pearson Educación, 2001. ISBN: 968-444-419-2